

■ Components

Interview Q&A; — Topic 3

10 Questions • Beginner → Expert

● Beginner

● Intermediate

● Expert

React Interview Prep • Topic 3: Components & Props • Akshay Chavhan

Interview Questions & Model Answers

1

Beginner

Q1. What is a React component?

→ A component is an independent, reusable piece of UI. It's a function (or class) that accepts props as input and returns JSX describing what the UI should look like. React apps are built by composing many components into a tree.

■ *Interview Tip: Use the analogy: YouTube page = Navbar + VideoPlayer + Comments, each is a component.*

2

Beginner

Q2. What is the difference between Functional and Class components?

→ Functional components are plain JS functions that use Hooks for state and lifecycle. Class components extend `React.Component` and use `this.state` and lifecycle methods like `componentDidMount`. Functional components are now the recommended approach — simpler, less boilerplate, and Hooks are more powerful.

■ *Interview Tip: Always say 'Functional is recommended today' — shows you know the current best practice.*

3

Beginner

Q3. What are props? Can you modify them?

→ Props are inputs passed from a parent to a child component. They are read-only — a component must never modify its own props. This enforces one-way data flow and keeps components predictable. To send data back up, you pass a callback function as a prop.

■ *Interview Tip: One-way data flow is a core React principle. Mention it explicitly — interviewers love it.*

4

Beginner

Q4. What is the children prop?

→ `children` is a special built-in prop that represents whatever JSX is placed between a component's opening and closing tags. It's how you build wrapper/layout components like modals, cards, and containers without knowing what content will be inside them.

■ *Interview Tip: Give a real-world example: a Modal or Card component that wraps dynamic content using children.*

5

Intermediate

Q5. Why must component names start with a capital letter?

→ React uses capitalization to distinguish between custom components and native HTML elements. A lowercase tag like `map` maps to an HTML element. A capitalized tag like `Map` tells React to call the corresponding function and render its output as JSX.

■ *Interview Tip: This links directly to how `React.createElement()` works — lowercase = string, uppercase = function reference.*

6

Intermediate

Q6. What is component composition?

→ Composition is building complex UIs by combining smaller components together, rather than extending them through inheritance. React strongly favors composition — you pass components as props or children to create flexible, reusable designs. The React team explicitly says they've never found a use case for component inheritance.

■ *Interview Tip: Contrast with Angular or Java OOP inheritance patterns — shows architectural awareness.*

7

Intermediate

Q7. What are default props and how do you set them?

→ Default props are fallback values used when a prop is not passed by the parent. In functional components you set defaults via destructuring: `function Button({ label = 'Click Me' })`. In class components you use `Component.defaultProps = {}`. In TypeScript projects, default parameter values in function signatures are preferred.

■ *Interview Tip: Mention both approaches (destructuring defaults vs `defaultProps`) to show full awareness.*

8

Intermediate

Q8. What are PropTypes and when would you use them?

→ PropTypes is a runtime type-checking library for React props. You define expected types for each prop and React warns in development if the wrong type is passed. In TypeScript projects, interfaces and types replace PropTypes entirely — they catch errors at compile time, not runtime, which is better.

■ *Interview Tip: If asked in a TS project context, say: 'We use TypeScript interfaces instead — compile-time is safer than runtime.'*

9

Expert

Q9. What is the mental model for a React component?

→ A component is a pure function: `UI = f(props)`. Given the same props, it always returns the same JSX. This makes components predictable, testable, and easy to reason about. When props change, React re-renders the component to produce updated UI.

■ *Interview Tip: `UI = f(state, props)` is the full formula — adding state shows deeper understanding.*

10

Expert

Q10. What is the difference between a controlled and uncontrolled component?

→ A controlled component's form data is driven by React state — React is the single source of truth. An uncontrolled component stores its own state internally in the DOM, accessed via refs. Controlled components are preferred because they give full control over form data for validation, transformation, and submission.

■ *Interview Tip: This is a very common interview question. Always explain with a form input example.*